

1. Preprocessor

- **Definition:** The preprocessor is a **tool that runs before the compiler**. It processes directives that start with **#** (like **#include**, **#define**, **#ifdef**) before actual compilation begins.
- **Role:** Prepares the code for the compiler by **including headers, defining constants, and conditional compilation**.
- **Key Points:**
 1. Works **before compilation**.
 2. Handles directives starting with **#**.
 3. Does **not check syntax** of C++ code.
 4. Examples:

```
#include <iostream> // includes the iostream header file
#define PI 3.14159 // defines a constant
```

2. Compiler

- **Definition:** The compiler translates the **preprocessed C++ code** into **machine code (binary/executable)** that the computer can run.
- **Role:** Checks **syntax and semantics**, optimizes code, and generates an **executable program**.
- **Key Points:**
 1. Works **after the preprocessor**.
 2. Checks for **syntax errors**.
 3. Converts human-readable C++ code to **machine language**.
 4. Example: Compiling a file **program.cpp** generates **program.exe** (on Windows) or **a.out** (on Linux/Mac).

3. Quick Comparison Table

Feature	Preprocessor	Compiler
When it runs	Before compilation	After preprocessing
Purpose	Prepares source code	Converts code to machine code
Syntax checking	No	Yes
Handles	Directives (#include , #define)	C++ code
Output	Modified source code	Executable/machine code

💡 Example Flow in C++ Compilation:

1. Source code → Preprocessor (`#include` files, `#define` constants handled)
 2. Preprocessed code → Compiler (syntax check + machine code generation)
 3. Machine code → Linker → Executable
-